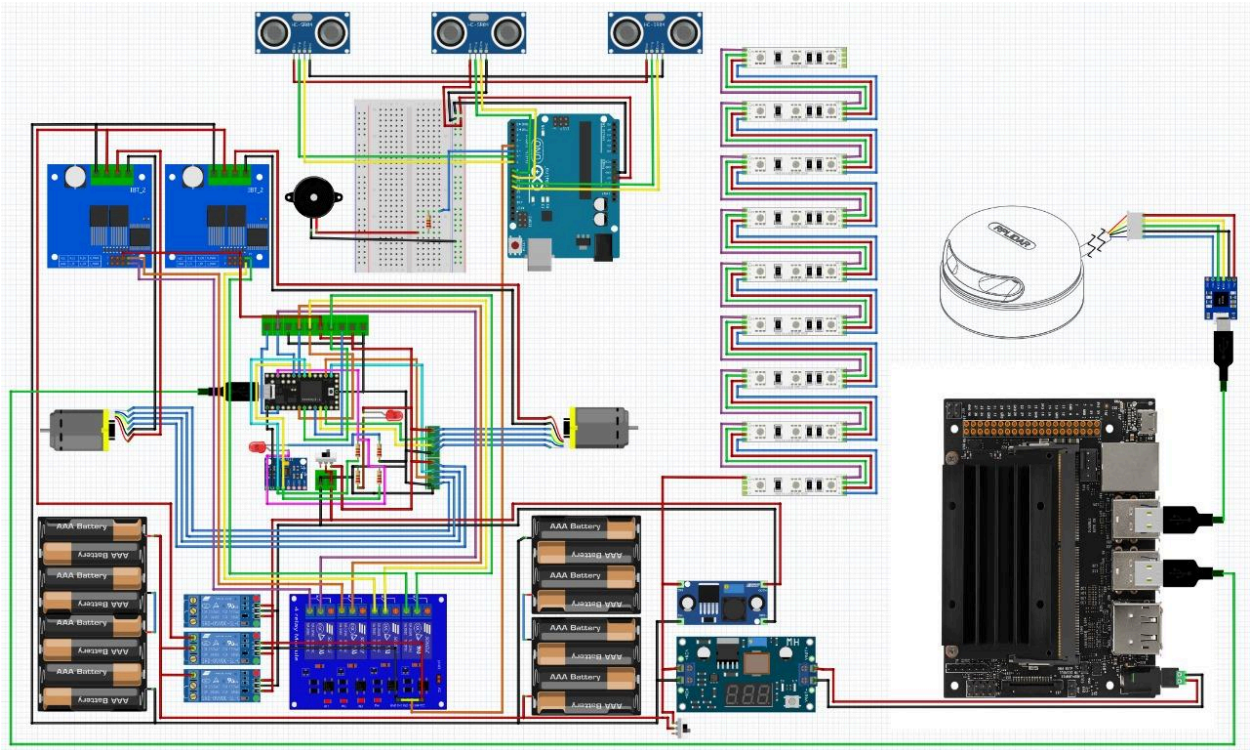
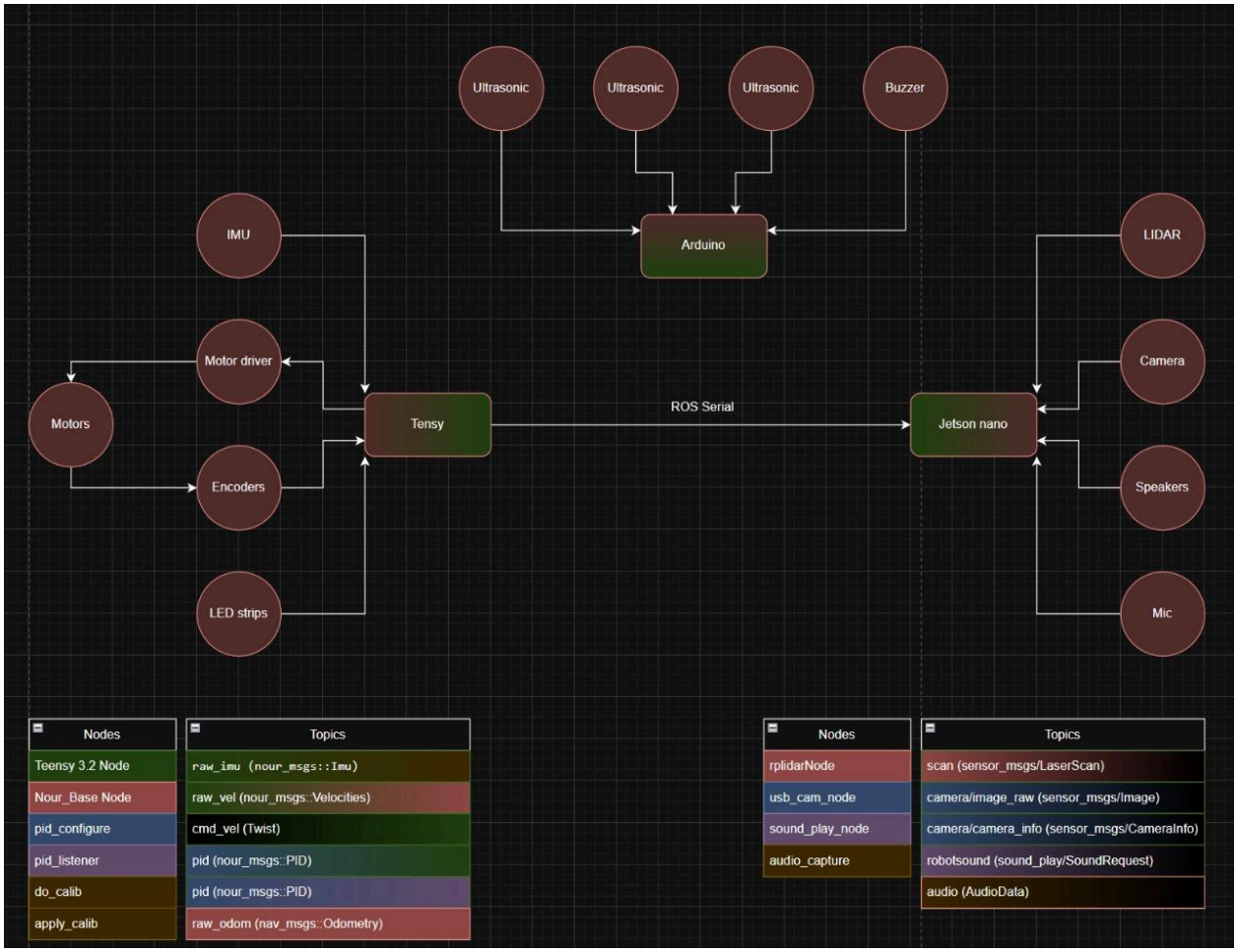
Circuit diagram



Nodes and Topics diagram



The nodes table specify the nodes names and the topics table specify the topics names in this format [topic_name (msg_name)]

Each node have a color
And each topic have two colors one for the publishing node (left color) and one for the receiving node (right color)

🔧 Hardware Overview

🔹 Microcontrollers & SBCs:

- **Teensy (3.2)**: Controls core robot hardware (motors, encoders, IMU, LED strips)
- **Arduino**: Handles ultrasonic sensors and buzzer
- **Jetson Nano**: Main processing hub running ROS, connects everything together

🔹 Peripherals Connected to Each Unit:

Teensy

- **Motors** (through motor driver)
- **Encoders**
- **IMU**
- **LED strips**

Arduino

- 3× **Ultrasonic sensors**
- **Buzzer**

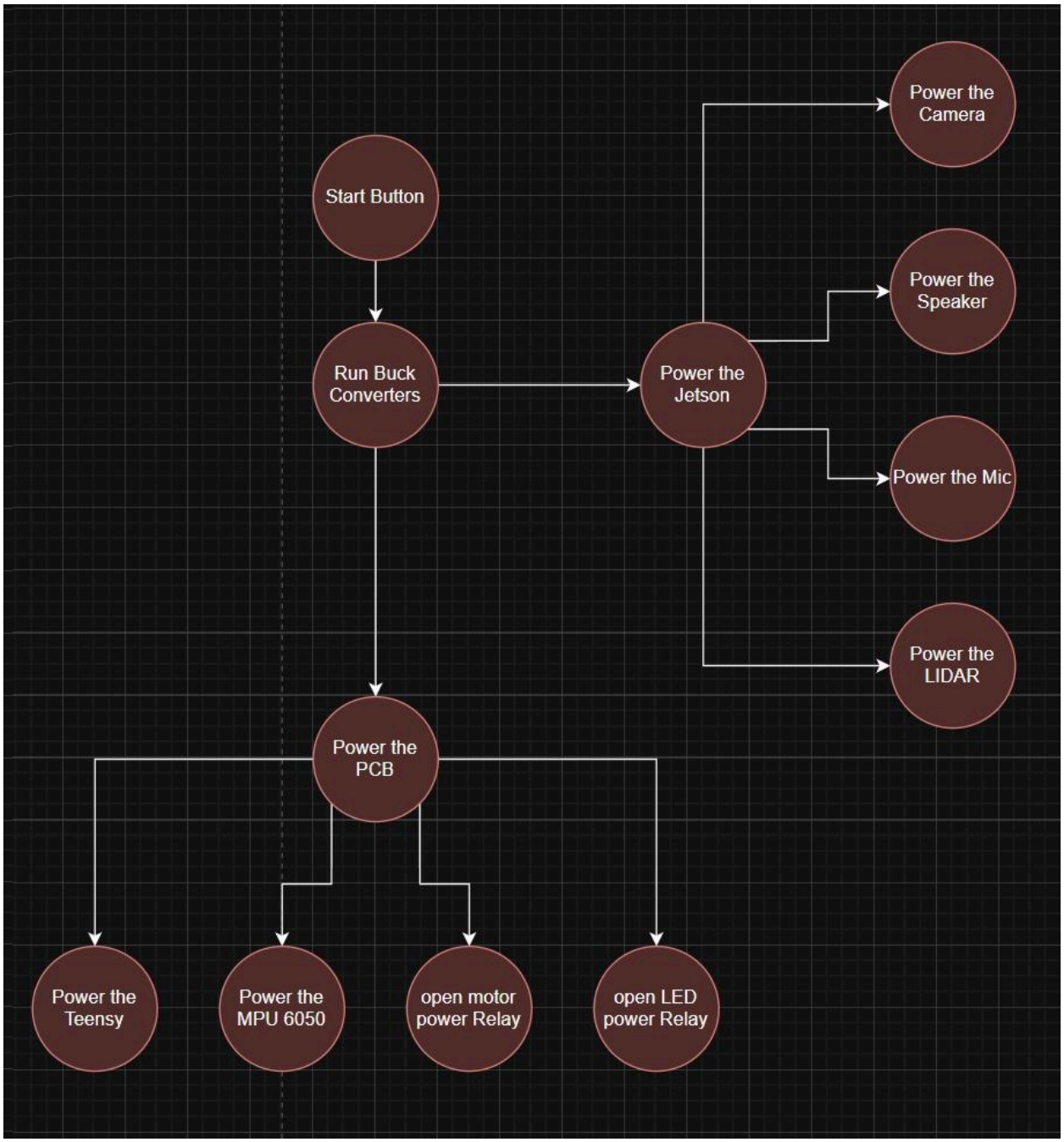
Jetson Nano

- **LIDAR**
- **Camera**
- **Speakers**
- **Mic**

🔁 Communication

- **ROS Serial** between Teensy/Arduino → Jetson Nano

Single Button Boot



This is a **power management flow diagram** for a robotic system that uses **a single Start Button** to intelligently boot up all hardware components in a sequenced and controlled manner. Here's a breakdown:

🔘 Startup Sequence (Single Button Boot)

1. Start Button Pressed

- User triggers the system by pressing one button.

2. Run Buck Converters

- Buck converters step down voltage from the main power source to appropriate levels (e.g., 12V to 5V/3.3V).
- This is the first step in distributing power.

⚡ Parallel Power Distribution

🔷 Branch 1: Jetson Nano & High-Level Peripherals

- **Power the Jetson Nano**
    - This also powers:
        - **Camera**
        - **Speaker**
        - **Mic**
        - **LiDAR**

These are high-power components, usually powered via USB or dedicated regulators from the Jetson side.

🔷 Branch 2: PCB & Core Control System

- **Power the PCB**
    - Activates control electronics, including:
        - **Teensy** (controls motors, encoders, LED, etc.)
        - **MPU6050** (IMU)
        - **Motor Power Relay**
        - **LED Power Relay**

This lets you **individually switch on high-current components** like motors and LED strips **via relays**, giving better power safety and control.

Motor Driver Replacement

To address the failure in our previous motor driver, we replaced the damaged component with **two new motor driver units**. This decision allowed us to balance current loads across the drivers, enhancing both **performance and reliability**. The new configuration supports higher current ratings.

Changes Implemented:

- **Pin Reassignment**: The codebase was updated to accommodate the **new dual driver layout**, ensuring proper PWM and direction signal mapping.
- **Code Refactor**: Refactored motor control logic in the Teensy firmware to manage dual motor drivers consistently and avoid redundant operations.

Teensy Repair and Recovery

The system experienced frequent errors due to **corrupted firmware and hardware on the Teensy microcontroller**.

Steps Taken:

1. **Issue Detection**: Fixed the Teensy hardware wiring problem
2. **Troubleshooting**: we found the library that caused error while uploading the code
3. **Validation**: system stability was restored and sustained over extended testing cycles.

PID Constants Optimization

Initial testing revealed the robot's motion was **unstable and oscillatory**, traced to improper PID control parameters for motor speed regulation.

Tuning Process:

- Conducted systematic tuning of **Kp (Proportional)**, **Ki (Integral)**, and **Kd (Derivative)** values using test ramps and step inputs.

- Final values achieved a good balance between **responsiveness and stability**, minimizing oscillation and improving tracking.

Project Achievements and Next Steps

With our hardware and control systems stabilized, we reached **90% completion** of our intended functionality. Future achievements include:

- **Bluetooth-Enabled Speaker Integration**: Successfully test speakers paired via Bluetooth dongle, adding voice feedback capability.

- **RealSense Camera Setup**: Power up and interface the RealSense depth camera, while handling its **high power demand** by adjusting power allocation.

- **Microphone Integration**: Mount and connect the microphone, test in various placements for optimal sound pickup, and validate data acquisition for potential sound-based triggers.

Arduino Emergency Stop System

To ensure safety and reliable obstacle detection, we addressed multiple issues affecting the **Arduino-based emergency stop system**. This system is crucial for triggering immediate motor shutdowns if obstacles are detected within unsafe proximity.

Issues Identified:

- **Damaged Ultrasonic Sensors**: ultrasonic units were no longer responsive.
- **Power Supply Instability**: The Arduino was inconsistently powered due to loose connections.
- **Faulty Wiring**: Signal lines were prone to noise and false readings.

Fixes and Enhancements:

1. **Sensor Replacement**: Installed new, calibrated ultrasonic sensors to replace the damaged units.
2. **Wiring Overhaul**: Cleaned up and secured all sensor signal and power lines using shorter, shielded cables where needed.
3. **Stable Power Source**: The Arduino is now reliably powered from a dedicated 5V rail on the PCB with reverse polarity protection.
4. **Functional Testing**: Conducted close-range and long-range detection tests. Verified emergency stop triggers using test objects and dynamic robot scenarios.

The system now reliably **halts motor commands** via interrupt signals sent to the Relayes when an obstacle is detected within the set threshold, significantly improving operational safety.